Strix

PENETRATION TEST REPORT

Security Assessment

/home/joan.belles/tabler-react

TARGET	/home/joan.belles/tabler-react
RUN	tabler-react_03a0
SCAN MODE	deep
STATUS	completed
STARTED	2026-10-29 16:26 UTC
COMPLETED	2026-10-29 18:06 UTC
DURATION	1h 40m 1s

CONFIDENTIAL

Executive Summary

0	0	0	1
CRITICAL	HIGH	MEDIUM	LOW

1 total findings across this assessment.

El análisis de la rama `feature/upgrade-vite` confirmó un hallazgo de **severidad baja**: destinos `javascript:` en props de navegación de componentes React pueden ejecutar JavaScript en el origen de una aplicación consumidora cuando usa React 18 y la víctima activa el enlace. No se demostró acceso a datos restringidos ni impacto sistémico; la exposición depende de cómo cada consumidor alimente los props. No se hallaron CVEs en los lockfiles ni secretos en el árbol de trabajo de esta rama. El riesgo prioritario es validar los destinos `href` y `to` en todos los enlaces y componentes de navegación personalizados, sin depender de la versión React elegida por cada consumidor.

Postura general: riesgo limitado, condicionado a la integración consumidora. No se validó ninguna cadena hacia acceso masivo a datos, compromiso del pipeline o ejecución en el servidor del producto.

Methodology

Evaluación **white-box** limitada exclusivamente a la rama `feature/upgrade-vite`. Se mapearon la biblioteca publicada, sus componentes y exports, la SPA de demostración, los scripts npm/Vite y ambos lockfiles; se distinguieron las superficies de runtime de las herramientas de desarrollo.

Se realizaron análisis estático y estructural sobre el repositorio, revisión de secretos y dependencias, y validación dinámica aislada de los enlaces con React/ReactDOM 18.3.1 y 19.3.1 en Chromium. También se construyeron la biblioteca y la demo; `npm run test:react18` pasó en una copia temporal, incluidas la compilación, la comprobación de tipos y las 132 pruebas del paquete. No se modificó el código fuente del checkout y la copia temporal de validación se eliminó.

El producto es una biblioteca UI, no un servicio backend: no se encontraron rutas de producto, base de datos ni persistencia propia. La demo usa datos ficticios y enlaza su servidor local a loopback.

Technical Analysis

Hallazgo confirmado — XSS mediante destinos `javascript:`: varios componentes exportados pasan `href` o `to` sin validar a anchors nativos o componentes de navegación que terminan renderizando anchors. La reproducción con React 18.3.1 confirmó ejecución al hacer clic en `BlogCard`, `SocialNetworksList` en modo enlaces, `Tag`, `Tag.AddOn`, `AlertLink`, `Nav.Link`, `Button`,

`ListGroupItem`, `PricingCardButton`, `GalleryCard.Image`, `GalleryCard.IconItem`, `NavBarItem`, `NavBarDropDown`/`MenuLink` y `DropdownItem`; `NavSubmenuItem` solo cuando recibe un `RootComponent` de routing. React 19.3.1 bloqueó las variantes probadas, pero el paquete también admite React 18. La ejecución demostrada cambia el estado del documento en el mismo origen; requiere que la integración pase una URL controlable por el atacante y que la víctima active el enlace. La calificación es **Low, CVSS 3.5** (`AV:N/AC:L/PR:L/UI:R/S:U/C:N/I:L/A:N`). El informe contiene las rutas, la evidencia y una corrección verificada para los componentes iniciales; las ubicaciones añadidas después son informativas porque no se ejecutó una matriz completa de regresión del fix para cada una.

Superficies comprobadas sin hallazgos confirmados: el análisis no encontró sinks explícitos de HTML crudo o evaluación de JavaScript en el runtime. Trivy y `npm audit` no reportaron CVEs en los dos lockfiles. Gitleaks no encontró secretos en el árbol vigente; las coincidencias históricas se refieren a commits fuera de la ascendencia de la rama solicitada y no se inspeccionaron porque quedan fuera del alcance.

Seguimientos abiertos: las props de avatar y `backgroundImage` pueden introducir CSS durante SSR e hidratación y producir un overlay visual, pero no se demostró una fuente atacante en un consumidor SSR real; no se clasifica como XSS ni se reporta. El generador `demo:docs` puede escribir fuera de su directorio generado si se añade un alias exportado con traversal, pero solo se ejecuta mediante un comando manual; no se confirmó un workflow externo que lo ejecute sobre contribuciones no confiables. Quedan anchors de `SiteLogo`, `SiteHeader` y otros componentes sin validación dinámica. La validez de patrones de credenciales detectados en commits ajenos a la rama también queda sin verificar. No se confirmó una cadena entre estos candidatos y el XSS reportado.

Recommendations

Prioridad inmediata

- Aplicar una política común de validación de esquemas a `href` y `to` antes de entregarlos a anchors nativos, `Link` o cualquier `RootComponent`. Permitir solo rutas relativas y esquemas necesarios como `http:`, `https:`, `mailto:` y `tel:`; rechazar destinos inválidos o ejecutables.
- Completar la regresión para todas las ubicaciones reportadas. Probar React 18 y React 19, anchors nativos y routing personalizado, variantes con mayúsculas/espacios y otros esquemas peligrosos, y confirmar que los destinos legítimos sigan funcionando.

Corto plazo

- Revisar las props de avatar y estilo en una aplicación consumidora SSR que acepte datos controlados por usuarios; mantenerlas como brecha pendiente hasta demostrar o descartar ese flujo.
- Añadir una comprobación de contención de rutas en `demo:docs` antes de escribir y confirmar que ningún pipeline lo ejecute en contribuciones no confiables.
- Validar los anchors restantes y, si se amplía el alcance más allá de `feature/upgrade-vite`, revisar la autenticidad y vigencia de las coincidencias históricas de secretos.

Retest: repetir la prueba de XSS en React 18/19 después de aplicar la corrección y cerrar por separado los seguimientos pendientes con evidencia de sus controles o de sus rutas de entrada.

Findings

1. XSS mediante destinos `javascript:` en enlaces de componentes compatibles con React 18

LOW

CVSS 3.5 **Target** Paquete npm `tabler-react` **Endpoint** src/components/BlogCard/BlogCard.react.tsx; src/components/SocialNetworksList/SocialNetworksList.react.tsx; src/components/Tag/Tag.react.tsx; src/components/Tag/TagAddOn.react.tsx; src/components/Alert/AlertLink.react.tsx; src/components/Nav/NavLink.react.tsx; src/components/Nav/NavSubmenuItem.react.tsx; src/components/Button/Button.react.tsx; src/components/List/ListGroupItem.react.tsx; src/components/PricingCard/PricingCardButton.react.tsx; src/components/GalleryCard/GalleryCardImage.react.tsx; src/components/GalleryCard/GalleryCardIconItem.react.tsx; src/components/NavBar/NavBarMenu.tsx; src/components/NavBar/NavBarItem.tsx; src/components/NavBar/NavBarDropDown.tsx; src/components/Dropdown/Dropdown.react.tsx; src/components/Dropdown/DropdownItem.react.tsx

DESCRIPTION

Varios componentes exportados aceptan destinos de navegación y los pasan a `href` o `to` sin validar el esquema. En aplicaciones consumidoras que usan la versión React 18 admitida por el paquete, una URL `javascript:` suministrada mediante esos props ejecuta código al activar el enlace; React 19 bloquea estos destinos, pero no cubre a consumidores con React 18.

IMPACT

Chromium ejecutó JavaScript en el mismo origen de la aplicación consumidora al activar enlaces renderizados por `BlogCard`, `SocialNetworksList`, `Tag`, `Tag.AddOn`, `AlertLink`, `Nav.Link`, `Button`, `ListGroupItem`, `NavSubmenuItem` con un `RootComponent` de routing, `PricingCardButton`, `GalleryCard.Image`, `GalleryCard.IconItem`, `NavBarItem`, `NavBarDropDown`/`MenuLink` y `DropdownItem` bajo React/ReactDOM 18.3.1. La prueba cambió el título o un marcador del documento; no demostró acceso a información restringida ni modificación persistente. El efecto requiere que una integración del consumidor pase una URL controlable por atacante y que la víctima active el enlace.

TECHNICAL ANALYSIS

El paquete admite React/ReactDOM `^18.0.0 || ^19.0.0`. En React 18.3.1, los anchors nativos de `BlogCard`, `SocialNetworksList`, `Tag` y `Tag.AddOn` conservan el valor `javascript:` en `href`. Validadores independientes confirmaron también: `AlertLink` (`src/components/Alert/AlertLink.react.tsx:16`); las ramas nativa y `RootComponent` de `Nav.Link` (`src/components/Nav/NavLink.react.tsx:41,45`); las ramas anchor/custom de `Button` (`src/components/Button/Button.react.tsx:157,164`); `ListGroupItem` nativo/custom (`src/components/List/ListGroupItem.react.tsx:37,46`); `NavSubmenuItem` únicamente al proporcionar un `RootComponent` de routing (`src/components/Nav/NavSubmenuItem.react.tsx:23-25`); `PricingCardButton` (`src/components/PricingCard/PricingCardButton.react.tsx:41,45,55`); `GalleryCard.Image` (`src/components/GalleryCard/GalleryCardImage.react.tsx:35,40,44`); `GalleryCard.IconItem` (`src/components/GalleryCard/GalleryCardIconItem.react.tsx:42,46,50`); `NavBarMenu`

→ `NavBarItem` (`src/components/NavBar/NavBarMenu.tsx:20`; `src/components/NavBar/NavBarItem.tsx:23-24`); `NavBarDropDown` → `Dropdown.Item`/`MenuLink` → React Router `NavLink` (`src/components/NavBar/NavBarDropDown.tsx:19-24,10-14`); y `Dropdown` → `DropdownItem` (`src/components/Dropdown/Dropdown.react.tsx:160-161`; `src/components/Dropdown/DropdownItem.react.tsx:78,82`). Las variantes personalizadas pasan `to` a un componente que termina en anchor, por lo que validar solo un `<a href>` nativo no es suficiente: hay que validar `href` y `to` antes de entregarlos a cualquier `RootComponent` o `Link`. `NavSubmenuItem` con su root por defecto solo recibe un atributo `to`, que Chromium no trata como navegación; la ejecución se confirmó para el `RootComponent` de routing. React 19.3.1 bloqueó los esquemas en las rutas probadas, pero el paquete admite React 18. No se incluyen `SocialNetworksList.asButtons` ni rutas `<img src>`, que fueron controladas sin ejecución; `SiteLogo`, `SiteHeader` y otros enlaces que siguen en cobertura abierta tampoco se incluyen.

PROOF OF CONCEPT

1. Integre el paquete en una aplicación consumidora con React y ReactDOM 18.3.1, versión admitida por el manifiesto.
2. Pase `javascript:void(document.title='executed')` a un destino de enlace confirmado, por ejemplo `Tag.href`, `AlertLink.href`, `Nav.Link.to`, `ListGroupItem.to`, `NavSubmenuItem.to` con `RootComponent={Link}`, `Button` con `RootComponent="a"` y `href`, `PricingCardButton.href`, `GalleryCard.Image.href`/`to`, `GalleryCard.IconItem.href`/`to`, `NavBarMenu.items[].to` o `DropdownItem.to`.
3. Para las variantes de navegación personalizadas verificadas, use un `RootComponent` que genere un anchor o React Router `Link`/`NavLink`; para el menú de navegación, use `NavBarDropDown` con subitems que incluyan `to`.
4. Sirva la vista renderizada por SSR y active el enlace en Chromium; observe el cambio del título del documento o del marcador JavaScript al valor `executed`.

POC SCRIPT

````
```javascript
const React = require("react");
const { renderToStaticMarkup } = require("react-dom/server");
const { Tag } = require("tabler-react");

const destination = "javascript:void(document.title='executed')";
const html = renderToStaticMarkup(
React.createElement(Tag, { link: true, href: destination }, "Abrir")
);
console.log(html);
```

Con React/ReactDOM 18.3.1, el HTML conserva el destino en `href`; la validación Chromium confirmó que al activar el enlace el título cambia a `executed`. Los validadores independientes confirmaron la misma ejecución en las demás variantes indicadas en los pasos.
````

EVIDENCE

````
Con React/ReactDOM 18.3.1, SSR conservó el destino y Chromium ejecutó el código al hacer clic. Ejemplo observado:

```html
<a class="tag expanded" href="javascript:void(document.title='executed')">Abrir</a>
````
````

```

El resultado del navegador fue `document.title = "executed"`. Las validaciones independientes confirmaron ejecución para los componentes nativos y los wrappers React Router incluidos en el análisis, entre ellos `AlertLink`, `Nav.Link`, `Button`, `ListGroupItem`, `NavSubmenuItem` con root de routing, `PricingCardButton`, `GalleryCard.Image`/`IconItem`, `NavBarItem`, `NavBarDropDown`/`MenuLink` y `DropdownItem`. React/ReactDOM 19.3.1 bloqueó los destinos en las variantes probadas. `SocialNetworksList.asButtons` y los atributos `<img src>` sirvieron como controles negativos sin ejecución.
```

REMEDIATION

Valide los destinos de navegación con una política común inmediatamente antes de entregarlos a un anchor nativo o a cualquier `RootComponent`/`Link` que pueda producir un anchor. Acepte rutas relativas y únicamente los esquemas necesarios, como `http:`, `https:`, `mailto:` y `tel:`; rechace o elimine el destino cuando el parseo falle o el esquema no esté permitido. Aplique la política tanto a `href` como a `to` en todas las variantes de los componentes listados, incluidos los wrappers de React Router, `MenuLink`, `NavLink` y `itemsObject`; no dependa de la versión de React instalada por el consumidor.